

Reglas de Navegación en JSF



Lección: Reglas de Navegación en JSF (Jakarta Faces)

1. Concepto Básico

En **JSF (Jakarta Faces)**, las reglas de navegación permiten definir cómo la aplicación transiciona entre diferentes páginas en respuesta a acciones del usuario o condiciones de la lógica de negocio.

Hay dos enfoques principales:

- **Navegación Implícita:** Determina la siguiente página con base en el nombre retornado por la acción del bean.
- **Navegación Explícita:** Utiliza el archivo `faces-config.xml` para centralizar las reglas de navegación.

2. Diferencia entre Managed Beans y Backing Beans

- **Managed Bean:**
 - Es un bean administrado por el contenedor JSF, anotado con `@Named` y un alcance (`@RequestScoped`, `@SessionScoped`, etc.).
 - Puede contener lógica de negocio y manejar eventos.
- **Backing Bean:**
 - Es un tipo específico de Managed Bean que se asocia directamente a una página JSF.

- Su propósito principal es manejar componentes y eventos específicos de la interfaz de usuario.

En el ejemplo que realizaremos:

- `Candidato` es un **Managed Bean** porque almacena los datos del candidato.
- `VacanteForm` es un **Backing Bean** porque maneja la lógica asociada al formulario de la vista.

3. Sintaxis Básica de las Reglas de Navegación en JSF

Managed Bean (`Candidato.java`):

```
package beans.model;

import jakarta.enterprise.context.SessionScoped;
import jakarta.inject.Named;
import java.io.Serializable;

@Named("candidato")
@SessionScoped
public class Candidato implements Serializable{
    private String nombre = "Valor";

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }
}
```

Backing Bean (`VacanteForm.java`):

```
package beans.backing;

import beans.model.Candidato;
import jakarta.enterprise.context.RequestScoped;
import jakarta.inject.Inject;
import jakarta.inject.Named;

@Named
@RequestScoped
public class VacanteForm {
```

```

@Inject
private Candidato candidato;

public void setCandidato(Candidato candidato) {
    this.candidato = candidato;
}

public String enviar() {
    if (this.candidato.getNombre().equals("Juan")) {
        return "exito?faces-redirect=true";
    } else {
        return "fallo?faces-redirect=true";
    }
}
}

```

4. Ejemplo Documentado de Reglas de Navegación Implícita

Página Inicial (index.xhtml):

```

<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html">
<h:head>
    <title>Reglas de Navegación con JSF</title>
</h:head>
<h:body>
    <h:form>
        <table>
            <tr>
                <td><h:outputLabel for="nombre" value="Nombre:"></h:outputLabel></td>
                <td><h:inputText id="nombre"
                    value="#{candidato.nombre}"></h:inputText></td>
                <td><h:message for="nombre"></h:message></td>
            </tr>
        </table>
        <h:commandButton action="#{vacanteForm.enviar}" value="Enviar" />
    </h:form>
</h:body>
</html>

```

Página de Éxito (exito.xhtml):

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html">
<h:head>
  <title>Éxito</title>
</h:head>
<h:body>
  <h:form>
    Gracias #{candidato.nombre} por aplicar a la vacante.
    <br/>
    <h:commandLink action="index?faces-redirect=true">Regresar</h:commandLink>
  </h:form>
</h:body>
</html>
```

Página de Fallo (fallo.xhtml):

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:h="jakarta.faces.html">
<h:head>
  <title>Fallo</title>
</h:head>
<h:body>
  <h:form>
    Gracias #{candidato.nombre}, no hay vacantes por el momento.
    <br/>
    <h:commandLink action="index?faces-redirect=true">Regresar</h:commandLink>
  </h:form>
</h:body>
</html>
```

5. Explicación del Ejemplo

1. Flujo de la Aplicación:

- La página inicial (index.xhtml) contiene un formulario que permite al usuario ingresar su nombre.
- Al presionar el botón "Enviar", se invoca el método `enviar` del Backing Bean `VacanteForm`.
- Dependiendo del nombre ingresado:
 - Si el nombre es "Juan", se redirige a `exito.xhtml`.
 - En cualquier otro caso, se redirige a `fallo.xhtml`.

2. Navegación Implícita:

- No se utiliza un archivo `faces-config.xml`. En su lugar, la navegación se basa en el valor retornado por el método del Backing Bean.
- Las vistas (`exito.xhtml`, `fallo.xhtml`) deben estar en el mismo directorio que `index.xhtml`.

3. Interacción entre Beans:

- El Backing Bean `VacanteForm` utiliza el Managed Bean `Candidato` para obtener el nombre ingresado por el usuario.

6. Mejores Prácticas

- **Alcance Adecuado:**

- Usa `@SessionScoped` si necesitas que los datos del usuario persistan entre múltiples páginas.

Con esta lección y el ejemplo presentado, puedes implementar reglas de navegación claras y funcionales en tus aplicaciones JSF. ¡Adelante con la práctica! 🚀

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)